

WEB SYSTEM DESIGN PROJECT – 2



VISTULA
UNIVERSITY

Name: Amjad Massaoud

Student ID: 78706

Course: Web Systems Design

Semester: 6th semester 57DPH

Date: 30.08.2026

Step 1 — Open the existing project	2
Step 2 — Verify Composer and autoload context	2
Step 3 — Explain PSR-12 before writing code	3
Step 4 — Create the controller file	3
Step 5 — Register the new route in routes/api.php	4
PSR-4 Explanation	4
Step 6 — Start the backend server and test the endpoint in the browser	5
Step 7 — Inspect the route list	6

Step 8 — Make one improvement to the JSON response	6
Step 9 — Add extension: accept POST too and add ping endpoint	7
Step 10 — Commit the changes to Git	9
1.1- Why controller-based routing is better than putting everything in a route closure	10
1.2- CAP / PACELC reference for API design	11

Step 1 — Open the existing project

I opened the project and went into the backend folder here.

```
jad@fedora-2:~$ cd uni-project/project2
jad@fedora-2:~/uni-project/project2$ cd backend
jad@fedora-2:~/uni-project/project2/backend$
```

Step 2 — Verify Composer and autoload context

I ran `composer -V` to check Composer, then opened `backend/composer.json` and saw the autoload section. Laravel already sets up PSR-4, which links namespaces to folders and class names to file paths.

```
jad@fedora-2:~/uni-project/project2/backend$ composer -V
Composer version 2.9.5 2026-08-30 11:40:53
PHP version 8.5.9 (/usr/bin/php85)
Run the "diagnose" command to get more detailed diagnostics output.
```

composer.json

```
1      "autoload": {
2          "psr-4": {
3              "App\\": "app/",
4              "Database\\Factories\\": "database/factories/",
5              "Database\\Seeders\\": "database/seeders/"
6          }
7      },
8
9
```

Step 3 — Explain PSR-12 before writing code

PSR-12 is a PHP coding standard that builds on older style guides. It defines rules for things like indentation, spacing, and how code should be structured. The main goal is to make code consistent and easy to read, especially when multiple developers are working on the same project. PSR-12 helps prevent confusion and makes it easier to maintain and understand the code.

Step 4 — Create the controller file

I created `EchoController.php` in `app/Http/Controllers`. The controller reads a message query parameter from the request and returns it as a JSON response. If no message is provided, it uses a default value. Using a controller keeps the logic organized and separate from the routes, making the code easier to maintain and more readable.

- **`declare(strict_types=1);`** is good modern PHP practice and fits well with PSR-12 examples.
- **`namespace App\Http\Controllers;`** tells PHP and Composer that this class belongs to the `App\Http\Controllers` namespace, which Laravel maps through Composer PSR-4 autoloading into `app/Http/Controllers/`.
- **`use Illuminate\Http\Request;`** imports the request class that gives access to query parameters and input.
- **`use Illuminate\Http\JsonResponse;`** makes the return type explicit and readable.
- **`public function echo(...)`** defines the controller action.
- **`$request->query('message', 'Hello from EchoController')`** means if the request contains ?message=something, use that, otherwise use the default text.
- **`response()->json([...])`** sends structured JSON back to the client.

```
1  <?php
2
3  declare(strict_types=1);
4
5  namespace App\Http\Controllers;
6
7  use Illuminate\Http\Request;
8  use Illuminate\Http\JsonResponse;
9
10 class EchoController
11 {
12     public function echo(Request $request): JsonResponse
13     {
14         $message = $request->query('message', 'Hello from EchoController');
15
16         return response()->json([
17             'message' => $message
18         ]);
19     }
20 }
21
```

Step 5 — Register the new route in routes/api.php

I opened routes/api.php and registered a new route for the Echo API. I added a GET route for /echo that is handled by the echo method in EchoController. Laravel maps the URL to the controller automatically using PSR-4 autoloading, so the class is found correctly. Since the route is in api.php, Laravel adds the /api prefix, making the full URL http://127.0.0.1:8000/api/echo. Using a controller keeps the logic organized instead of putting it directly in the route.

PSR-4 Explanation:

PSR-4 is a PHP standard for autoloading classes using namespaces. It maps a PHP namespace to a folder in the project. For example, the namespace App\Http\Controllers points to the folder app/Http/Controllers. This means when Laravel sees EchoController::class, Composer knows exactly which file to load (EchoController.php). PSR-4 ensures that class names match file paths in a case-sensitive way, making the project organized and easy to maintain.

api.php

backend > routes > api.php

```
1  <?php
2
3  use App\Http\Controllers\EchoController;
4  use Illuminate\Support\Facades\Route;
5
6  Route::get('/echo', [EchoController::class, 'echo']);
```

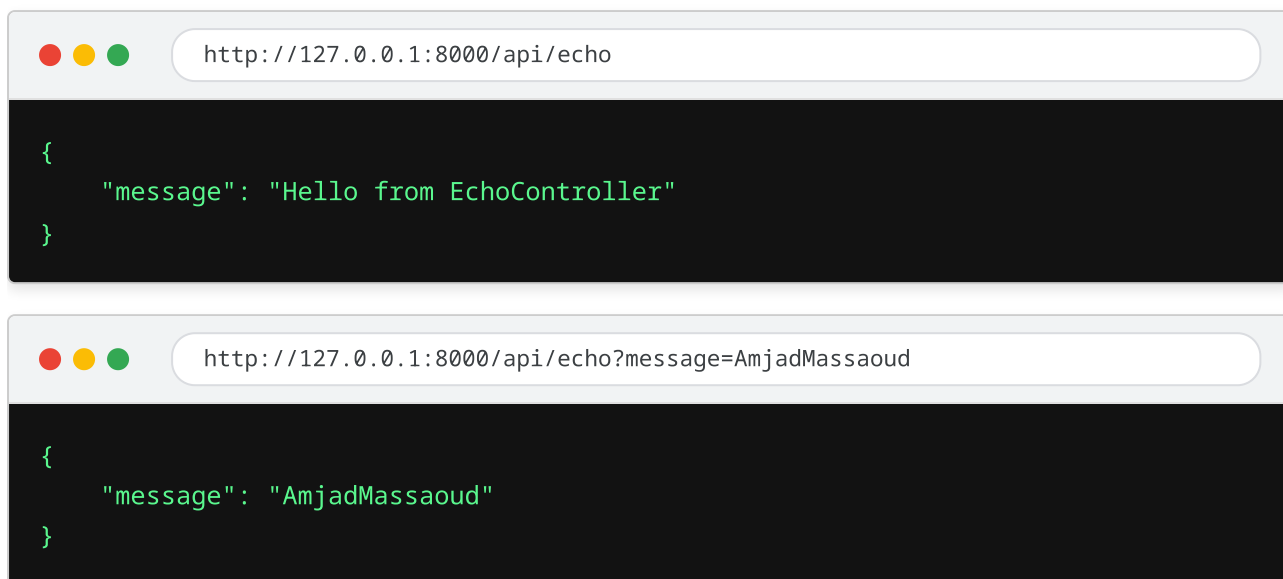
Step 6 — Start the backend server and test the endpoint in the browser

I ran `php artisan serve` to start Laravel locally at `http://127.0.0.1:8000`. I opened `http://127.0.0.1:8000/api/echo` in the browser and saw the JSON response. Then I tested it with a query parameter (`?message=NameSurname`) and the controller returned my message correctly.

```
jad@fedora-2:~/uni-project/project2/backend$ php artisan serve
```

```
INFO Server running on [http://127.0.0.1:8000].
```

```
Press Ctrl+C to stop the server
```



Step 7 — Inspect the route list

I ran `php artisan route:list` to verify that Laravel recognized the new route. In the list, I found `GET|HEAD api/echo` pointing to `EchoController@echo`, which confirmed that the route file was read correctly and Laravel could resolve the controller.

```
jad@fedora-2:~/uni-project/project2/backend$ php artisan route:list
GET|HEAD / .....
GET|HEAD api/echo ..... EchoController@echo
POST api/echo ..... EchoController@echo
GET|HEAD api/echo/ping ..... EchoController@ping
GET|HEAD health .....
GET|HEAD storage/{path} ..... storage.local
PUT storage/{path} ..... storage.local.upload
GET|HEAD up .....

Showing [8] routes
```

Step 8 — Make one improvement to the JSON response

I updated the echo method in EchoController to return more information for API testing. Now, when I visit `http://127.0.0.1:8000/api/echo?message=Test123`, the JSON response includes not only the message, but also the HTTP request method and the current server timestamp. This makes the endpoint more useful for testing purposes.

EchoController.php

```
15         return response()->json([
16             'success' => true,
17             'message' => $message,
18             'method' => $request->method(),
19             'timestamp' => now(),
20         ]);
21     }
```

`http://127.0.0.1:8000/api/echo?message=Test123`

```
{"success":true,"message":"Test123","method":"GET","timestamp":"2026-08-30T14:51:44.199739Z"}
```

Step 9 — Add extension: accept POST too and add ping endpoint

I added a new POST route and an additional ping endpoint in `routes/api.php` pointing to EchoController. I updated the controller so it reads the message from both the query string (GET) and the request body (POST). Then I tested the endpoints using Postman for both GET and POST requests. The JSON response correctly returned the message along with the HTTP method and timestamp in both cases.

Postman API Testing - POST Route

My Workspace / echo-test

POST http://127.0.0.1:8000/api/echo

Body • Pretty • JSON

Status: 200 OK • Time: 342 ms • Size: 325 B

```
{
  "success": true,
  "message": "Hello from EchoController",
  "method": "POST",
  "timestamp": "2026-08-30 15:06:52"
}
```

Postman API Testing - GET Custom Ping Route

My Workspace / echo-test

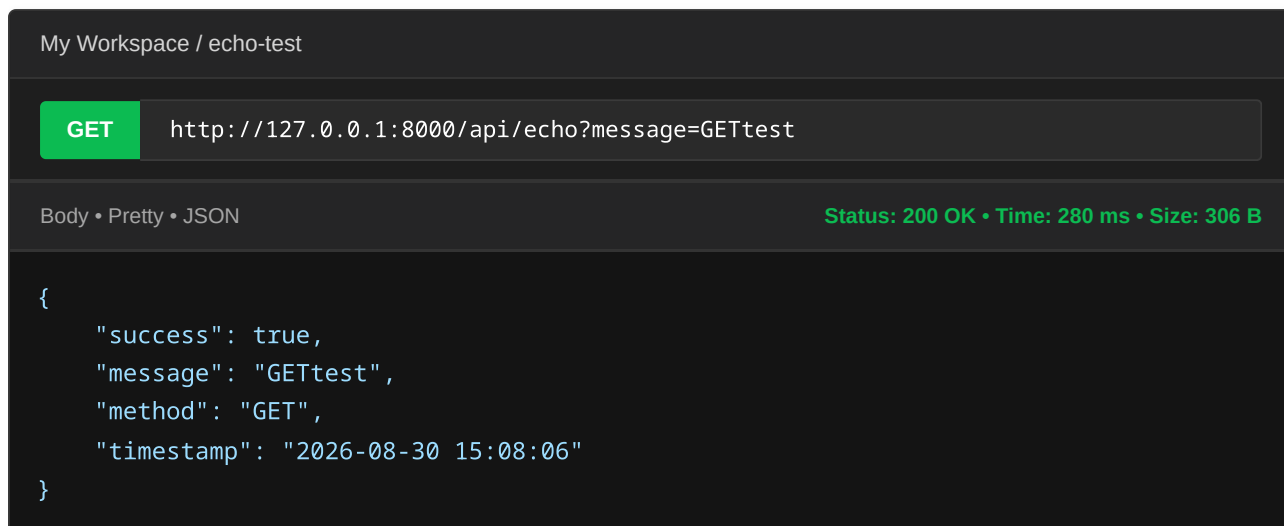
GET http://127.0.0.1:8000/api/echo/ping

Body • Pretty • JSON

Status: 200 OK • Time: 185 ms • Size: 260 B

```
{
  "success": true,
  "status": "service-online",
  "timestamp": "2026-08-30 15:07:18"
}
```

Postman API Testing - GET Custom Parameter



Step 10 — Commit the changes to Git

Once everything was working, I saved my work in Git. I ran `git status` to see which files had changed, which included `EchoController.php` and `routes/api.php`. Then I staged the changes with `git add .`, committed them with the message `Project 2: add PSR-4 controller routing with /api/echo`, and pushed them to the remote repository.

```

jad@fedora-2:~/uni-project/project2$ git add .

jad@fedora-2:~/uni-project/project2$ git commit -m "Project 2: add PSR-4
controller routing with /api/echo"
[main 88c2b21] Project 2: add PSR-4 controller routing with /api/echo
2 files changed, 28 insertions(+), 1 deletion(-)

jad@fedora-2:~/uni-project/project2$ git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 24 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 533 bytes | 266.00 KiB/s, done.
Total 5 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/amjadmassaoud/wsd-project.git
43c0160..88c2b21 main -> main

```

Project 2: add PSR-4 controller routing with /api/echo

amjadmassaoud committed on Aug 30, 2026 • 3 commits

 backend	Project 2: add PSR-4 controller routing with /api/echo	30.08.2026
 frontend	Project 1: scaffold Laravel backend and Vue frontend	30.08.2026
 .gitignore	Project 1: scaffold Laravel backend and Vue frontend	30.08.2026

1.1- Why controller-based routing is better than putting everything in a route closure

Controller-based routing separates the application's logic from the route definitions. This makes the code **more organized, easier to read, and maintainable**, especially in larger projects. Controllers allow **reusable methods** across multiple routes and make **unit testing** easier. In contrast, putting all logic directly in route closures can quickly make the code messy and hard to manage as the project grows.

1.2- CAP / PACELC reference for API design

I learned from CAP and PACELC that in distributed systems you often have to make trade-offs. For an API, this could mean choosing between data consistency, speed, and availability, or deciding what happens if the network has issues. Using a controller-based and organized API helps manage these trade-offs more easily. The JSON responses stay consistent, and the endpoints work quickly, making the API predictable and efficient.